GameUp Platform - 베타테스트 클라우드 배포 계획

> **작성일**: 2026-04-08
> **목표 도메인**: `https://www.gameup.co.kr/betatest`
> **목적**: 내부 베타테스트 환경 구축
> **상태**: 계획 수립 완료 - CSP/도메인 정보 확인 후 실행 예정

Context

GameUp Platform을 `https://www.gameup.co.kr/betatest` 경로에서 내부 베타테스트 용도로 배포합니다. 현재 Docker, CI/CD, 클라우드 설정이 전혀 없는 상태이므로 처음부터 구성합니다.

선택된 기술 스택: - **클라우드**: Naver Cloud Platform - **컨테이너**: Docker Compose (단일 서버) - **CI/CD**: GitHub Actions - **리버스 프록시**: Nginx - **SSL**: Let's Encrypt (certbot)

사전 확인 필요 사항

실행 전 아래 정보를 확인해야 합니다:

- [ ] **Naver Cloud 계정 정보** (콘솔 접속 가능 여부)
- [ ] **도메인 등록기관** (가비아, 호스팅케이알 등) 및 DNS 관리 접속 정보
- [ ] **www.gameup.co.kr 도메인 소유 확인** 및 기존 DNS 레코드 상태
- [ ] **Kakao/Naver OAuth** Developer Console 접속 권한
- [ ] **Toss Payments** 시크릿 키 (테스트 또는 실서비스)

Phase 1: 소스코드 수정 (basePath 대응)

`/betatest` 경로 prefix 하에서 동작하도록 소스코드 수정이 필요합니다.

1.1 `apps/web/next.config.ts` 수정

- `basePath: '/betatest'` 추가
- `output: 'standalone'` 추가 (Docker 최적화)
- `images.remotePatterns` 에 프로덕션 도메인 추가
- `rewrites` 를 개발환경 전용으로 조건 분기 (프로덕션에서는 Nginx가 처리)

1.2 `apps/web/src/services/api.ts` 수정

- `API_BASE_URL` 을 `NEXT_PUBLIC_API_URL` 환경변수로 제어
- 하드코딩된 `/api/auth/session` 경로를 basePath 인식하도록 수정

1.3 프론트엔드 경로 감사

- `<a href="/...">`, `window.location`, `fetch('/api/...')` 등 Next.js `<Link>` 밖에서 하드코딩된 경로를 검색하여 수정
- Socket.io 클라이언트 경로 확인 (`/betatest/socket.io/`)

1.4 OAuth 콜백 URL 업데이트

- Kakao Developer Console 콜백 URL: `https://www.gameup.co.kr/betatest/api/auth/callback/kakao`
- Naver Developer Center 콜백 URL: `https://www.gameup.co.kr/betatest/api/auth/callback/naver`

Phase 2: Docker화

2.1 `.dockerignore` 생성

```
node_modules
.next
dist
.turbo
.git
e2e/
docs/
.env*
apps/api/uploads/*
playwright-report/
```

2.2 `apps/api/Dockerfile` 생성

- **Base**: `node:22-alpine`, 멀티스테이지 빌드
- pnpm workspace 의존성 설치 후 `tsx` 로 실행 (workspace TS 참조 문제 회피)
- uploads 디렉토리를 볼륨 마운트 포인트로 설정
- **포트**: 5000

2.3 `apps/web/Dockerfile` 생성

- `next build` → standalone 출력 사용
- 빌드타임에 `basePath`, `NEXTAUTH_URL`, `API_URL` 환경변수 주입

- **포트**: 3000

2.4 `docker-compose.yml` 서비스 구성

서비스	이미지/빌드	포트	비고
mongodb	`mongo:7.0`	내부전용	`wiredTigerCacheSizeGB: 1`, 볼륨 마운트
api	`./apps/api/Dockerfile`	내부전용	mongodb 의존, uploads 볼륨
web	`./apps/web/Dockerfile`	내부전용	api 의존
nginx	`nginx:alpine`	80, 443	리버스 프록시, SSL 종료

Docker Volumes: `mongodb_data`, `uploads_data`, `certbot_data`, `certbot_www`
Network: `gameup-net` (bridge)

2.5 Nginx 프록시 라우팅

```
/betatest/api/*       → http://api:5000/api/*         (prefix strip)
/betatest/uploads/*   → http://api:5000/uploads/*      (prefix strip + 캐싱)
/betatest/socket.io/* → http://api:5000/socket.io/*    (WebSocket upgrade)
/betatest/*           → http://web:3000/betatest/*     (Next.js, basePath 포함)
```

추가 설정: - HTTPS 강제 리다이렉트 - Let's Encrypt ACME challenge 경로 (`.well-known/acme-challenge/`) - `client_max_body_size 100M` (파일 업로드 대응) - 정적 자산(`_next/static`) 장기 캐싱 (365일)

Phase 3: Naver Cloud 서버 구성

3.1 서버 스펙 (권장)

항목	권장 사양
서비스	VPC > Server
타입	Standard s2-g3 (2 vCPU, 8GB RAM) 이상
OS	Ubuntu 22.04 LTS
스토리지	100GB SSD
리전	Korea (KR)

> 예산 여유 시 s2-g4 (4 vCPU, 16GB) 권장

3.2 ACG (보안 그룹) 설정

프로토콜	포트	소스	설명
TCP	22	사무실 IP만	SSH 접속
TCP	80	0.0.0.0/0	HTTP (HTTPS 리다이렉트용)
TCP	443	0.0.0.0/0	HTTPS

> 27017(MongoDB), 3000(Web), 5000(API)은 외부 노출하지 않음 - Docker 내부 네트워크에서만 통신

3.3 서버 초기 설정 순서

1. 시스템 패키지 업데이트
2. Docker Engine + Docker Compose v2 설치
3. `deploy` 사용자 생성 + Docker 그룹 추가
4. GitHub Actions용 SSH 키 생성 및 등록
5. UFW 방화벽 설정 (22, 80, 443만 허용)
6. Swap 2GB 설정 (메모리 안전장치)
7. 레포지토리 clone

Phase 4: 환경변수 및 시크릿

서버에 생성할 파일 (Git에 포함하지 않음)

`.env.api`:

```
PORT=5000
NODE_ENV=production
MONGODB_URI=mongodb://mongodb:27017/gameup
JWT_SECRET=<64자 랜덤 문자열 생성>
JWT_EXPIRES_IN=7d
ALLOWED_ORIGINS=https://www.gameup.co.kr
MAX_FILE_SIZE=104857600
TOSS_SECRET_KEY=<토스 시크릿 키>
```

`.env.web`:

```
NEXTAUTH_SECRET=<64자 랜덤 문자열 생성>
NEXTAUTH_URL=https://www.gameup.co.kr/betatest
API_URL=http://api:5000
NEXT_PUBLIC_API_URL=/betatest/api
```

```
KAKAO_CLIENT_ID=<카카오 클라이언트 ID>
KAKAO_CLIENT_SECRET=<카카오 클라이언트 시크릿>
NAVER_CLIENT_ID=<네이버 클라이언트 ID>
NAVER_CLIENT_SECRET=<네이버 클라이언트 시크릿>
```

> `API_URL=http://api:5000` → 서버사이드 렌더링 시 Docker 내부 네트워크 통신
> `NEXT_PUBLIC_API_URL=/betatest/api` → 클라이언트 브라우저에서 API 호출 경로

GitHub Actions Secrets

Secret 이름	용도
`DEPLOY_HOST`	서버 공인 IP
`DEPLOY_USER`	`deploy`
`DEPLOY_SSH_KEY`	SSH 개인키

Phase 5: SSL/TLS (Let's Encrypt)

1. **1단계**: HTTP-only nginx 설정으로 먼저 컨테이너 시작
2. **2단계**: certbot으로 인증서 발급 `bash docker compose run --rm certbot certonly \ --webroot --webroot-path=/var/www/certbot \ -d www.gameup.co.kr -d gameup.co.kr \ --email admin@gameup.co.kr --agree-tos --no-eff-email`
3. **3단계**: SSL nginx 설정으로 전환 후 재시작
4. **자동 갱신**: 매월 1, 15일 cron으로 갱신 + nginx reload `bash 0 0 1,15 * * docker compose run --rm certbot renew && docker compose exec nginx nginx -s reload`

Phase 6: DNS 설정

도메인 등록기관에서 아래 레코드를 추가/수정합니다:

타입	호스트명	값	TTL
A	gameup.co.kr	`<서버 공인 IP>`	300
A	www	`<서버 공인 IP>`	300

> 초기 TTL 300초 (빠른 전파) → 안정화 후 3600초로 변경

Phase 7: GitHub Actions CI/CD

`.github/workflows/deploy-beta.yml` 생성

```yaml
name: Deploy Beta
on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  lint-and-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: 'pnpm'
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm type-check

  deploy:
    needs: lint-and-typecheck
    runs-on: ubuntu-latest
    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.DEPLOY_HOST }}
          username: ${{ secrets.DEPLOY_USER }}
          key: ${{ secrets.DEPLOY_SSH_KEY }}
          script: |
            cd /home/deploy/gameup
            git fetch origin main
            git reset --hard origin/main
            docker compose build
            docker compose up -d --remove-orphans
            sleep 10
            curl -sf http://localhost:5000/api/health || exit 1
            echo "Deploy successful"
```

Phase 8: 데이터 시딩 및 검증

초기 시딩

```
docker compose exec api npx tsx src/scripts/seed.ts
```

기존 시드 스크립트가 `upsert` 패턴이므로 멱등성 보장 (여러 번 실행해도 안전).

기본 테스트 계정

역할	이메일	비밀번호
Admin	admin@gameup.com	test123456
Developer	developer@test.com	test123456
Player	player@test.com	test123456

검증 체크리스트

- [ ] `https://www.gameup.co.kr/betatest` 접속 확인
- [ ] 로그인 (이메일 인증)
- [ ] 로그인 (카카오 OAuth)
- [ ] 로그인 (네이버 OAuth)
- [ ] 게임 목록 조회
- [ ] 커뮤니티 글 작성 + 이미지 업로드
- [ ] 관리자 패널 접근 (`/betatest/admin`)
- [ ] 개발자 콘솔 접근 (`/betatest/console`)
- [ ] 모바일 반응형 확인
- [ ] WebSocket 실시간 메시징
- [ ] 토스 결제 테스트 모드
- [ ] 파트너 매칭 기능

생성/수정 파일 목록

작업	파일 경로	설명
수정	`apps/web/next.config.ts`	basePath, standalone, rewrites 조건분기
수정	`apps/web/src/services/api.ts`	API URL basePath 대응
감사	프론트엔드 전체	하드코딩된 경로 수정
생성	`.dockerignore`	Docker 빌드 제외 파일
생성	`apps/api/Dockerfile`	API 서버 컨테이너
생성	`apps/web/Dockerfile`	Web 프론트엔드 컨테이너

작업	파일 경로	설명
생성	`docker-compose.yml`	전체 서비스 오케스트레이션
생성	`nginx/nginx.conf`	Nginx 메인 설정
생성	`nginx/conf.d/gameup.conf`	사이트별 프록시 설정
생성	`.github/workflows/deploy-beta.yml`	CI/CD 파이프라인
생성	`.env.example.api`	API 환경변수 템플릿
생성	`.env.example.web`	Web 환경변수 템플릿
생성	`scripts/server-setup.sh`	서버 초기 설정 스크립트
생성	`scripts/ssl-init.sh`	SSL 초기 발급 스크립트

주의사항

1. **basePath 파급효과**: Next.js `<Link>`는 자동 처리되지만, `<a>`, `fetch()`, `window.location` 등은 수동 수정 필요
2. **MongoDB 메모리**: 8GB 서버에서 `wiredTigerCacheSizeGB: 1`로 제한 필수
3. **볼륨 관리**: `docker compose down -v` 실행 시 DB/업로드 데이터 **삭제됨** - 주기적 백업 권장
4. **Workspace TS 참조**: API 컨테이너는 `tsx` 런타임으로 실행하여 빌드 복잡성 회피
5. **OAuth 콜백**: 카카오/네이버 개발자 콘솔에서 새 콜백 URL 등록 필수
6. **업로드 저장소**: 베타에서는 로컬 볼륨 사용, 프로덕션 전환 시 Naver Object Storage(S3 호환) 마이그레이션 고려

향후 프로덕션 전환 시 추가 고려사항

- MongoDB Atlas 또는 Naver Cloud MongoDB 매니지드 서비스로 전환
- Naver Object Storage로 파일 업로드 마이그레이션
- CDN 적용 (정적 자산)
- 로드밸런서 + 다중 서버 구성
- 모니터링 (Naver Cloud Monitoring, Grafana)
- 백업 자동화 (MongoDB dump + 업로드 파일)
- basePath 제거 후 루트 경로(`/`)에서 서비스